M.A.R.V.I.N.

One assistant, enforced discipline: a design for AI pair-programming that survives real projects

Robert Ilisei · October 2026 · v1.0.0 · github.com/RobertIlisei/MARVIN

M.A.R.V.I.N. — Moderately Advanced Robotic Virtual Intelligence Network. A pair-programming AI IDE for macOS.

Abstract

AI coding assistants perform impressively in demos and degrade steadily on real projects. The causes are structural, not model-quality: context that evaporates between sessions, agent architectures that amplify errors instead of containing them, and guardrails written as prompt suggestions the model is free to skim past. MARVIN is a working counter-proposal, built and used daily by its author on real production projects. It makes four bets that run against the current grain: **one** assistant moving through enforced phases instead of a team of agents; a **knowledge graph** consulted before any file is read; behavioral rules hardened into **enumerated contracts** — the most load-bearing of them enforced at the tool gate, below the prompt; and a strictly **local-first** architecture with no backend, no telemetry by default, and credentials stored nowhere but your own machine. This paper describes the design, the reasoning, the enforcement mechanisms, and the results — each labeled measured, estimated, or design property — including first-message context cut from ~566K tokens (an amount that no longer fit the model's window) to ~13.4K on the same mature project, a fresh tab's entire first request halved from 142,596 to 72,284 tokens once the fixed load was measured instead of assumed, and multi-day plans that survive session boundaries. MARVIN is open source and installs via Homebrew.

---

1. The problem: AI assistance decays with project age

Every developer who has used an AI assistant on a project past week two recognizes the pattern. The tool that wrote the first scaffolding flawlessly begins to contradict decisions it made a month ago. It re-explores the codebase from scratch each session, burning tokens on rediscovery. Feature ten quietly breaks an assumption feature three was built on — and neither the human nor the AI can hold the whole project in their head to notice.

Three structural failures drive the decay:

Context does not persist. A session's understanding — the architecture choices, the gotchas, the "we tried that and it broke" — dies with the session. The next conversation starts from zero, so long-lived projects pay a rediscovery tax that grows with the codebase, while the *reasons* behind decisions are lost entirely.

More agents amplify errors instead of catching them. The 2026 multi-agent evaluation literature reports what production users had already noticed: on sequential coding work, multi-agent pipelines degrade output quality by up to 70%, and flat "bag of agents" topologies amplify a single agent's error up to 17× as peers build on each other's mistakes.[1] Handoffs lose context; parallel writers race; a review agent rubber-stamps what a planner agent hallucinated. The industry's answer to "one AI makes mistakes" has been "add more AIs," and the evidence says that answer is wrong for code.

Guardrails are advisory. Most tools express safety and process rules in the system prompt: *please ask before deleting, please check the docs first.* Long prompts thin a model's attention; a rule stated on page three of a system prompt fires unreliably, and "use your judgement" degrades to "do whatever the sampling temperature suggests." When MARVIN's own development audited its prompt-only rules, a test-driven-development directive had fired **approximately zero times across thousands of qualifying turns**.[2] A rule that cannot be enforced is a hope, not a rule.

There is also a quieter fourth problem: **trust**. An agent with shell access on a developer's machine, talking to a cloud service, is a serious security surface. Developers are right to ask what it can touch, what it phones home, and where their credentials go.

---

2. Design position: four bets

MARVIN is a macOS IDE — file tree, editor, terminal, chat — wrapped around the Claude Agent SDK. Its interest to an engineer evaluating the field is not the pane layout; it is four architectural bets, each a deliberate rejection of a current default.

Bet 1 — One assistant with phases, not an agent team

MARVIN is a single Claude session in a continuous user↔assistant loop. The roles other tools distribute across agent teams — product owner, tech lead, implementer, QA — exist in MARVIN as **phases one assistant moves through**, in one conversation, with the user as continuous overwatch: intake → discovery → impact analysis → architecture → plan → implement → verify → ship. No handoffs between peers means no context loss at the seams and no 17× error amplification.[3]

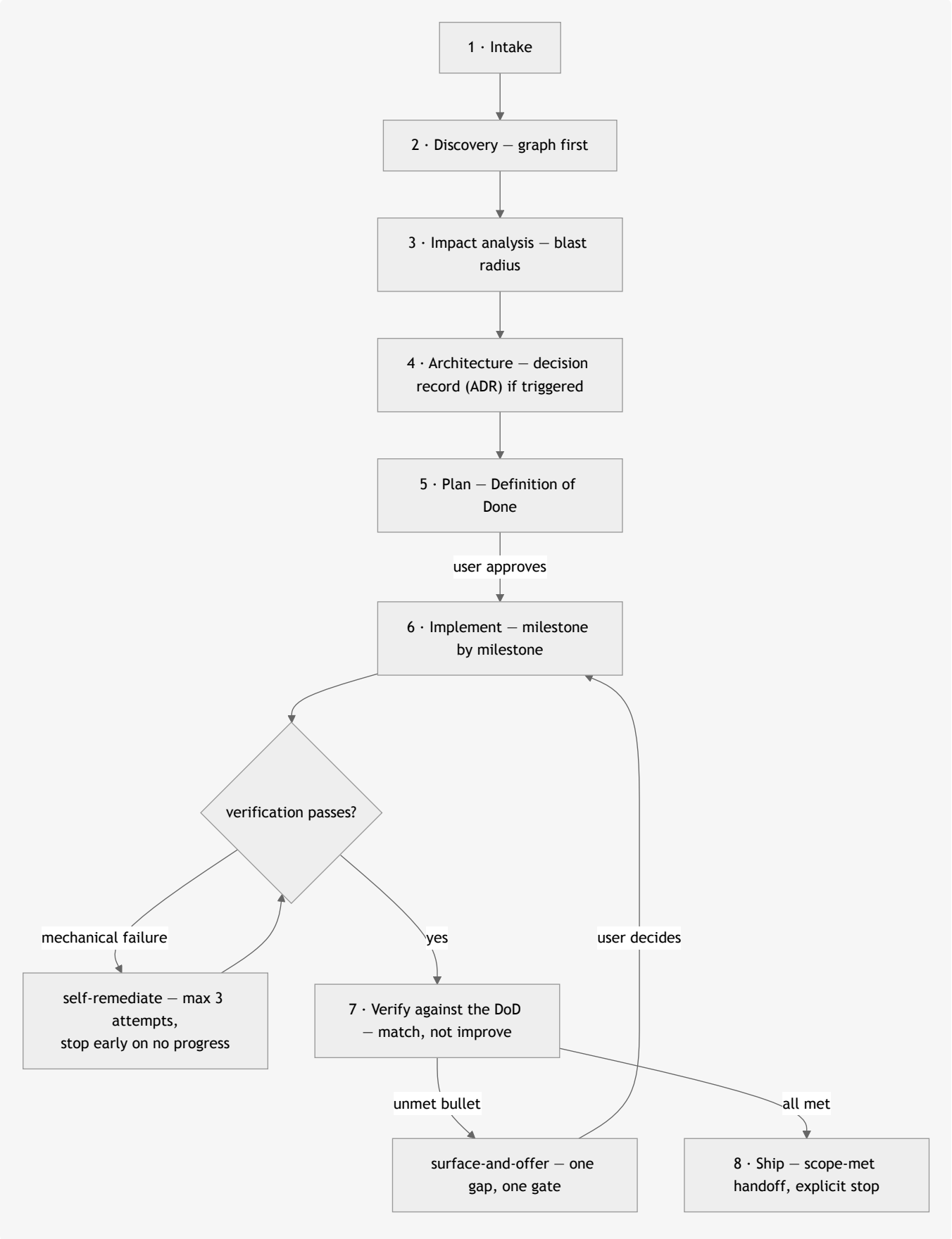


Figure 1 — one assistant through eight phases. The only loops are bounded: mechanical failures self-remediate at most three times (with an early stop when consecutive attempts produce identical errors), and scope-level gaps route through the user — never around them.

The exceptions are narrow, and read-only. Exactly three kinds of subagent are sanctioned — an **advisor** (a second opinion that stress-tests plans and architecture decision records), a **scout** (breadth-first read-only research), and read-only **audit fan-outs** — and all three are **unable to write**: the file-editing tools are denied to them at the SDK level, and the permission gate hard-denies any mutating tool call carrying a subagent ID — including shell commands its classifier judges mutating.[4] Parallel *reading* scales fine. Parallel *implementation* is the failure mode the literature documents, and MARVIN forbids it at the gate, not in prose.

Several loops, not several agents

One assistant per conversation does not mean one conversation. A person can run several MARVIN tabs at once, each on its own git worktree and branch, each steered by them — that is this topology multiplied, not violated, because no model dispatches another. The tabs are genuinely independent: since v0.1.110 each carries its own autonomy mode, models, effort, voice and permission posture, so a tab planning a refactor at maximum effort sits beside one answering questions read-only and cheaply.

Since ADR-0127 a person may also start a **supervisor** for a project: a session that watches their tabs and answers what is already decided — a question the plan answers, a nudge inside an approved plan, an ordinary permission card. It is the one place a model steers another, and it is bounded in code rather than in its prompt. It holds no write tools, and each category it acts in starts in shadow until the user switches it on. Everything on the human-only list becomes an item in an escalation inbox showing the raw act, which the user's click performs as theirs: integration, discarding work, boundary paths, a wider posture, scope, a new tab. Inside any turn it touched, those acts ask the user even in full auto. It opens a tab only on the user's request, in a fresh worktree; one request is one tab. Its append-only ledger is reviewed by the session auditor, whose report goes to the user and never to the supervisor.

Two things fall out of that which are worth stating plainly, because both were learned by being wrong about them.

A worktree isolates the filesystem, not the machine. Each tab gets its own checkout and branch. It does not get its own Docker daemon, host ports, databases, or anything keyed on `$HOME`. Three tabs running integration tests concurrently once shared a single reused Postgres container, and two suites that drop tables in setup corrupted each other. MARVIN ships no knowledge of any specific tool, so the remedy is a lever rather than a rule: a project declares the environment that makes its per-session runs independent, and MARVIN applies it to that tab's turns.

Removing a branch is only safe if something else holds its commits. On 2026-10-30 a tab's branch was deleted while its last two commits were reachable from nothing else. The supervisor had closed the tab with *keep*; the app, still showing it, asked a second time and sent *discard*; and a second tab, which had fast-forwarded onto the first and been named after its commit, took the freed name. Three small facts, each defensible alone. The fix is structural rather than a rule to be careful: any delete of a branch first pins the commits no other ref reaches under `refs/marvin/archive/` and refuses to delete if it cannot, a tab is never named after a sibling's commits, and a tree the server has already closed closes without asking.

An interface can be audited like a codebase. The native app was audited from source against a written product and design brief (9/20, 104 findings; then a critique, 25/40). The worst result was the least visible: five VoiceOver labels in fifty thousand lines. Every P1 was fixed, every P2 fixed or deferred with a reason, and the confirm that matters most now has one default behaviour — Allow is ⌘↵, Return consents to nothing.

A cost number is only as good as what it counts. Until 1.0 the cost panel overstated spend about 3.8×: the SDK's `total_cost_usd` is a running total for a resumed session and the tracker recorded it as one turn's cost (one real session: $93 recorded against a true $32). Each turn now records its own increase, and each figure is labelled by billing source — a plan's API-equivalent is not a bill, an API key is, an OpenRouter figure is an estimate. The plan name and usage windows come from the SDK's `accountInfo()`, and a window that has expired is not drawn, so no plan rule lives in the code.

Parallelism has a bill, and it is not the model's. On a metered CI runner, five tabs each opening their own merge request costs five full pipeline runs. Because a tab's branch is cut from the checkout's current HEAD, folding all five in locally costs none — the commits ride along in whatever run that branch was already going to have. MARVIN therefore batches integration by default and asks before anything opens a request or starts a pipeline, in every permission mode. Asking turned out not to be enough: two tabs each opened their own merge request after the confirm was allowed twice in a row, so since v0.1.113 a tab's own branch is never pushed or requested at all — the refusal names *Merge all* and *Prepare MR*, which fold the day's finished branches into one local commit and one request (ADR-0109, second amendment).

Isolation hides the shared state too. Two tabs once fixed the same defect on the same afternoon, byte-for-byte, one of them shipping a regression test the other never saw. Nothing was broken: the backlog lives in the project root, each worktree carries a copy frozen at the commit its branch was cut from, and the second tab read the copy. The remedy is the one Anthropic's own documentation gives for parallel workers — tasks are *claimed*, and "each teammate owns a different set of files" — applied to MARVIN's shape: an item is claimed by the tab working it, and the claim is visible in every listing and in the refusal a duplicate gets; a read of the worktree's copy is turned back at the gate in every permission mode; and when a tab resolves an item, the other tabs whose branch touches its files are told — as information marked as coming from another tab, never as an instruction from the user (ADR-0113).

A control has to look like a control. The same discipline applies to the interface itself, and it is measurable there too. Counted across MARVIN's seven left panes before any of them was redesigned: five button styles, five private section headers, four corner radii, and three different registers for the sentence a pane shows when it is empty. No single one is a bug; together they mean that a thing which looks the same does not behave the same, which is the one property an interface cannot afford to lose. The remedy was the same shape as everything else here — decide once, put the decision where it can be asserted, and let the drawing hold no rules. What made it worth doing was what it exposed on the way: a search result that opened the file but not the line it matched, a replace-all with no confirmation and no undo, and two toggles that were not controls at all, one of which loads tool servers into the model's reach (ADR-0114).

Nothing survives a tab unless it holds a commit. Measured on one project after a week of this: seventeen session worktrees, sixteen for closed tabs, twelve already merged, three empty, none ever swept. A closed tab now leaves a branch only when there is something on it; a dry-run merge reports

the first conflict before any merge is attempted; and the tab that owns a branch is the one asked to resolve it, because it knows what its changes were for (ADR-0111). "Merged" is itself derived, not recorded, and v0.1.115 widened what counts: a branch integrated by squash or rebase shares no commits with its target, so ancestry never saw it and 2.2 GB of such trees sat on one project marked *ready*. A branch now also counts as merged when merging it would change nothing — `git merge-tree` yields the target's own tree — and a checkout under MARVIN's worktree directory on a branch MARVIN did not name is adopted rather than left invisible to every sweep.

Many loops still share one machine, and one sidecar. A dozen tabs made MARVIN itself the bottleneck before the model was: a CPU profile put 4.9 s of every 25 s inside one status route, which ran ~150–200 synchronous `git` calls to derive every worktree's state, and every other request queued behind it. The derivation is now written once and driven two ways — synchronously for the rare mutation paths, asynchronously for every read — with status served stale-while-revalidate, and a decision that could discard a tree asks for it fresh. The same week the app leaked one chat model per split-view rebuild (68 alive after 64 rebuilds, each holding a stream open) until six streams exhausted the per-host connection pool and a transcript fetch queued forever. Both were found by measuring — a profile, a heap count — after reading the code had produced plausible, wrong explanations.

Bet 2 — The knowledge graph comes before the file read

For any structural question — *how does X work, who calls Y, what breaks if I change Z* — MARVIN queries a per-project knowledge graph **before** reading source files. Two graphs, actually: an AST-derived code graph (functions, types, calls, imports) and a knowledge graph of the project's documentation, architecture decisions, and memory. Both are rebuilt as you work — per turn, debounced, while the project is open — at zero LLM cost.[5] "Rebuilt automatically" is a claim that must be checked, and twice it was false without anyone noticing: the installed app never shipped the knowledge-graph builder, then ran it under a Python that could not import the graph library — one project's knowledge graph went fifteen hours stale while twenty decision records landed. MARVIN now finds the newest graph tool on the machine, runs the builder under that tool's own interpreter at low priority, spaces rebuilds at ten times the last run's duration, and says once, with the error, when a build fails.

The economics are the point. A graph query answering "what depends on this service" costs a fraction of the tokens of opening a dozen candidate files — measured at 27.5× cheaper per structural question — and catches couplings a keyword grep cannot see, because the graph encodes *edges*, not text matches. The discipline is graph-to-locate, files-to- verify: the graph points at two files out of hundreds; the assistant reads exactly those and cites `file:line`. "Grep and pray" is the failure mode this exists to eliminate.

Bet 3 — Deterministic contracts, not "use judgement"

Everywhere MARVIN's behavior matters, a soft rule was first replaced with an enumerated contract: a MUST-trigger list, a MUST-NOT list, and a narrow judgement test only for cases the lists don't cover. When to consult the graph. When a decision requires a written architecture record. When to spawn the advisor. When a skill fires. What may be written to memory. Each of these is a **firm surface** — auditable and testable.[6]

That form did its job and then became the problem. By September 2026 the prompt had grown to 29.4K tokens — 91 MUSTs, 43 MUST NOTs — and current models *over*-fire on emphatic trigger lists while following a short plain rule as well as a long one. Since v0.1.115 the shape is: **one plain rule in the prompt, the procedure in a skill loaded when it is needed, and a gate wherever the rule must hold regardless of what the model does.** The base prompt is ~4.4K tokens; the procedures ship as MARVIN's own skills (`marvin:adr`, `marvin:graph-tools`, …) that always match the running version; a test holds the parsed formats verbatim and the whole prompt under an 8K budget. Checked against the old prompt on the same three tasks, outcomes matched.[17]

The novel step is pushing contracts below the prompt entirely, into the runtime. MARVIN's tool gate can **deny a blind source-file read** when the graph should have been consulted first, **deny an edit in security-sensitive paths** until an advisor consult has happened, and — since v0.1.102 — **refuse a** `git commit` whose diff touches a security boundary until the review skills have actually run.[14] Since v0.1.115 the same gate also requires the change-impact report, which now looks past the call graph: three parallel tabs had each found the break their own commit caused — an enum the API spec never learned, a page calling a route by URL string, a restore script mounting a renamed config file — described it in a backlog item, and declared scope met. None had a call edge to see. The report now lists files outside the branch that *name* what the branch changed, and the scope-met check sends a parked item back when it touches the session's own changes: a break your change caused is part of the change.[18] The prompt asks; the gate *enforces*. This "design hooks" layer exists because measurement showed prompt-only rules decay with prompt length — so the most load-bearing rules stopped being prompt rules.

Honesty about the ladder: most of the contract families are still prompt-level. The enumerated MUST/MUST-NOT form measurably outperforms the soft language it replaced — that is what the zero-fires audit forced — but only gate-level rules are truly deterministic, which is why the direction of travel is downward: when a prompt contract proves load-bearing, it earns a runtime hook. Since v0.1.103 that promotion has a mechanism rather than a human reading transcripts: the **practice loop** (§4) mines a project's own sessions for the failures that repeat, proposes a rule, and an approved rule enforces at the same gate at one of three tiers.[15] The hooks themselves are configurable (enforce by default, measure-only, off); the hard-deny floor and the subagent invariant are not — no configuration weakens those two.[7]

The same philosophy governs scope. Before non-trivial work, MARVIN states a falsifiable **Definition of Done** (3–5 bullets an observer could mark yes/no) and verifies against exactly that — *match, not improve*. Mechanical failures (typecheck, tests, build) self-remediate, capped at three attempts with an early stop when consecutive attempts produce identical errors. Scope-level gaps are surfaced with a proposed next step and **gated on the user** — one gap, one gate. A fully autonomous retry-until-done loop was considered and deliberately rejected: it institutionalizes the "helpful spiral" of unrequested work that plagues agentic tools.[8]

Bet 4 — Local-first, or: there is no server

MARVIN has no backend. The app and its sidecar run on `localhost`; inference goes directly from the user's machine to Anthropic via the Claude Agent SDK. Precision matters here: inference necessarily carries prompt context — including the code MARVIN reads — to Anthropic, and that is the one network egress MARVIN *itself* makes. Tools acting on your behalf can create others, each passing

through the gate: a `git push` to your own remotes, a shell command you allow, the opt-in browser. Credentials are the user's own API key — an Anthropic Console key, or an OpenRouter key — stored in a `0600` file on the user's own disk, displayed only as its last four characters, and sent nowhere except the provider it belongs to.[9] There is no telemetry by default; the optional observability integration exports traces to the *user's own* Honeycomb account, configured by the user. Release artifacts are signed (minisign, with the public key pinned in three places across two repositories), so the supply chain from GitHub Release to `brew install` is verifiable.[10]

Project state follows the same principle: everything MARVIN learns about a project — its graphs, memory, decision records, plans, backlog — lives in *the project's own directory*, in plain text, in the user's repo. MARVIN holds no cross-project state and starts every new project from zero, by design. The user's data is the user's.

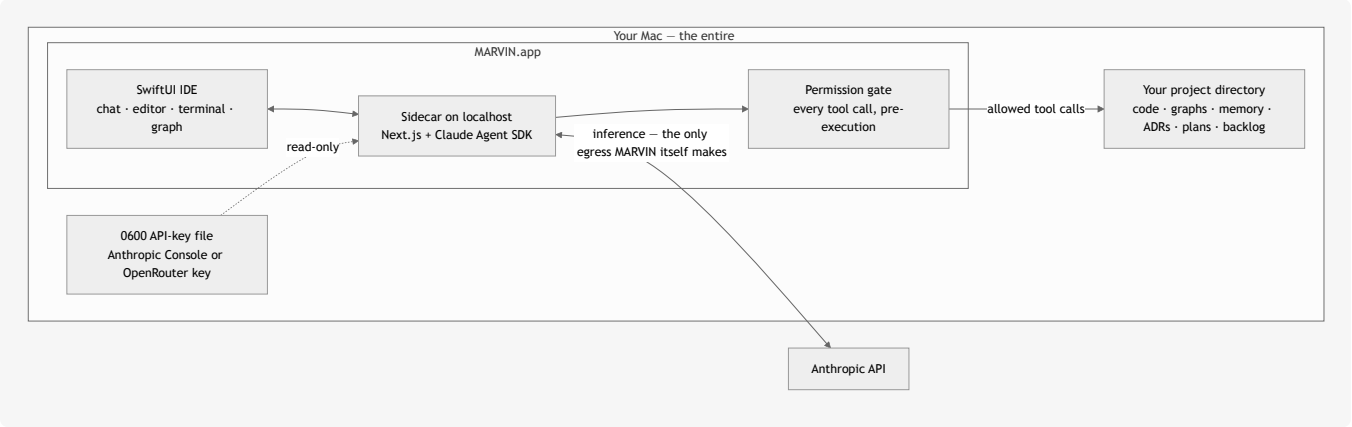


Figure 2 — there is no server. The only egress MARVIN itself makes is the inference call to Anthropic (which carries prompt context, including code it reads); tool-driven egress — a `git push` to your remotes, an approved shell command, the opt-in browser — happens on your behalf, through the gate. Credentials are read in place, never copied or forwarded; all project knowledge lives in the project's own repo.

---

3. How it works: a session

The four bets compose into a working loop. What follows is the actual shape of a session, not an idealization — the examples are drawn from real production transcripts, anonymized.[11]

Modes set autonomy; the gate confirms edits. Two orthogonal dials: *mode* (what MARVIN may attempt — read-only **Ask**, autonomous **Agent**, approval-gated **Plan**) and *permission strategy* (how each edit is confirmed — **auto**, per-diff **gated**, or **full auto**, which also waives the containment confirms a worktree tab raises when it reaches outside its own checkout, and nothing else: metered CI still asks, and the deny floor is untouched).[19] The out-of-the-box defaults are **Agent + auto** — near-full bypass, stated plainly because a security-minded reader should know it — with the hard-deny floor (destructive shell patterns, force-pushes to main, credential-file writes) short-circuiting in both strategies and every mutating call classified and written to an audit log regardless. Ask mode's read-only promise is enforced at the gate, not requested: the file-editing tools are denied outright, and shell commands pass the same mutating-or-not classifier that polices everything else.

Plan mode splits strategy from execution. A Plan turn runs read-only on the user's chosen *planner* model, presents a numbered plan grounded in the project's own decision history, and stops. An explicit approval runs it as a separate turn on the *executor* model. (Routing turns to different models does not add agents: the conversation stays one thread with one writer — "single assistant" is about loop topology, not model count.) The approved plan becomes a durable spine: it persists to a file, ticks off step by step, survives chat switches and app relaunches, and is re-injected into the model's context every turn so context compaction cannot make the assistant forget what it is executing. In a real multi-day session, a compliance-audit plan spanned dozens of turns; each "Continue" re-anchored MARVIN to *that plan and only that plan* — no re-auditing, no drift — while an advisor consult caught five defects in a decision record before it was ratified.[11]

Milestones are vertical slices. A plan for a change that touches more than one layer must make its first milestone the thinnest path that proves one flow through every layer it touches — the *tracer bullet* of *The Pragmatic Programmer* — verified end to end, with later milestones widening that path. The alternative, "database first, API next, frontend last", has the agent code each layer against nothing, integration surprises arrive last, and the user waits longest for the first thing they can see. The graph supplies the slice: the structural path from the entry symbol the user touches to the persistence symbol it reaches is the milestone's touchpoint list. MARVIN ships no notion of what a layer is; the practice loop (§4) reads a project's layers off its own graph — its directories and file names — and measures whether plans cross them.[16]

Every turn sees the same prompt, and a small one. A fresh tab on a mature project once sent 142,596 tokens on its first request and died compacting six times in eleven minutes — compaction frees messages, and the problem was everything else. Measuring it with the SDK's own context accounting found three faults size alone did not explain: the system prompt differed between turn 1 and turn 2, and a resumed turn's prompt *replaces* the first, so from turn 2 the model had no project context from MARVIN at all; "isolation mode" had never been on, so every turn also carried the user's Claude Code plugins, the project's instruction file a second time and another tool's memory; and the indexes (decision titles, memory, backlog) loaded whole. Now one prompt is recorded per session and everything per-turn — mode, plan, session tree — rides a reminder on the turn itself; only the user's skills load from Claude Code's settings; indexes load as their newest slice plus the tool that fetches the rest; three of MARVIN's own tool servers load on demand. The same first request is **72,284 tokens**.[17] Instruction files are read the way other agents write them: `CLAUDE.md` and/or `AGENTS.md`, `@path` imports expanded inside the project, a near-copy of one in the other skipped and reported, and a subdirectory's file surfaced when work first reaches it.[17]

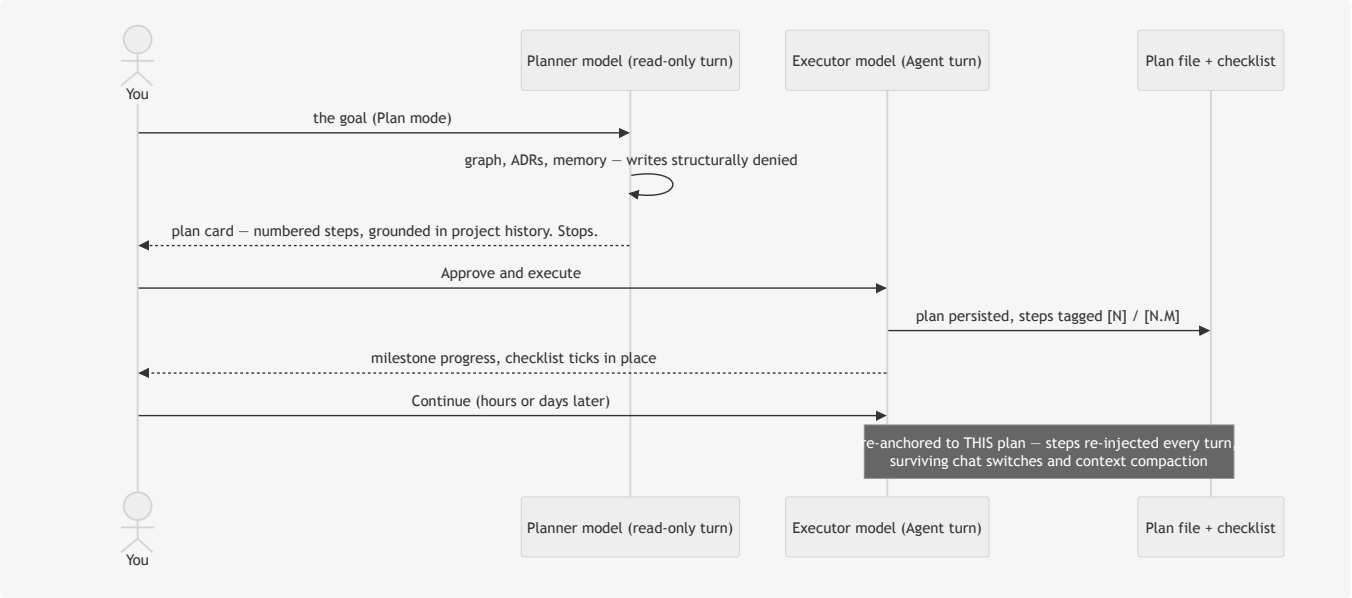


Figure 3 — planning and execution are different turns on different models. Approval is an explicit act; a paused plan resumes itself instead of ballooning into a re-audit.

Memory is curated, not logged. Cross-session persistence is split by content class, each with an enforced write path: durable *facts* (invariants, gotchas, constraints) go to a memory index via a tool that rejects status noise; deferred *work* goes to a backlog parking lot, consent-gated, never auto-executed; *decisions* go to architecture decision records; *status* goes to git. The design was forced by a real failure — an activity-logged memory file that bloated to 419KB, ~99% redundant, and overflowed the model's context window. The fix was not a bigger window; it was content-class enforcement at the write boundary.[1][2]

Curation has a second half, though: a store that only ever grows stops being read, and an unread backlog is indistinguishable from no backlog. So the same store gained a **review** pass — it reports near-duplicates that exact-title matching cannot see, auto-captures nobody triaged, items referencing files that no longer exist. Every one of those is a heuristic, which fixes the design: it *reports*, and the human decides. Nothing is resolved, merged or re-prioritised on the strength of a guess, because acting on a wrong heuristic deletes work nobody agreed to drop — the exact loss the backlog exists to prevent. The instruction travels with the data rather than only in the system prompt, so it survives being read out of context.[1][2]

That principle — *surface, never mutate* — is the one to carry across. Where an assistant's judgement is probabilistic, the safe boundary is between analysis and application: let it be freely wrong in what it notices, and structurally unable to act on being wrong.

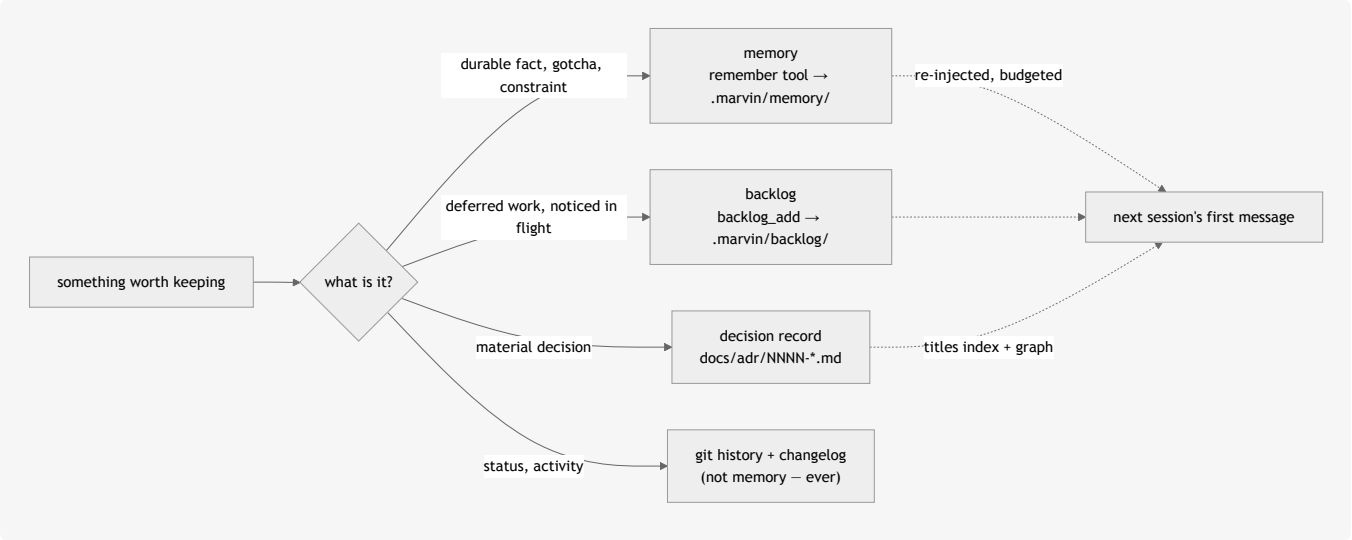


Figure 4 — persistence by content class, each with an enforced write path. The write tools reject the wrong class, which is what keeps the next session's context at ~13K tokens instead of 566K.

Work that outlives a turn is honest about it. Background jobs fire a real follow-up turn on process exit; scheduled wakeups re-invoke the assistant at a chosen time; and the prompt contract flatly forbids narrating a watcher that was never armed. The mechanism has to survive the machine, too: a twenty-hour session that looked stuck every fifteen minutes turned out, measured against the power log, to be the Mac asleep — 51 of 51 gaps began within seconds of a sleep — so a live turn now holds a power assertion; a search that silently read the SDK's never-closing stdin is refused at the gate with the fix; a job's completion is never dropped for waiting behind a long turn; and running jobs are kept in a ledger that a restart reconciles, so a crash reports "result unknown" instead of silence.[19] Every real-work turn ends the same way: the Definition of Done restated as past-tense facts, then an explicit stop — "*Anything else, or should I stop?*"

---

4. What it delivers

Each result below is labeled for what it is — **measured** (a number with a date and a method), **estimated** (engineering arithmetic, not a benchmark), or a **design property** (verifiable in source, not in a chart). All measurements are the author's own, on the author's projects; n=1 developer is the honest sample size, and the reader's evaluation should rest on the mechanisms, which are open source.

Context that no longer overflows (measured — and honest about its origin). The baseline was MARVIN's own earlier design: it injected every decision record plus an unbounded memory log, which on one mature production project reached ~~566K tokens (139 ADRs plus a 419KB memory file) and simply failed to fit the model's window. The redesign — ADR *index* instead of bodies, memory *tail* instead of log, knowledge graph serving details on demand — brought the same project to **~~13.4K tokens**.[13] Fixing a self-inflicted failure, yes; the claim is that the *fix is architectural* — an index-plus-graph design whose context stays budgeted as a project grows, instead of one that degrades back.

A fixed load that is measured every turn (measured). The ~13.4K figure above was the *project context*; the whole first request also carries the base prompt, the SDK's own preset, tool definitions and skills. On the same production project a fresh tab's first request went from **142,596 tokens to**

72,284 — a stable per-session prompt (+ real settings isolation), indexes as a recent slice, deferred tool servers, and the base prompt cut from 29.4K to ~4.4K. Every turn now logs its baseline, and a turn that starts over half its window says so.[17]

Structural questions at graph prices (estimated). The graph-first rule replaces open-ended file exploration with one ranked query plus targeted reads. This is now **measured, not estimated**: `graphify benchmark` on MARVIN's own repository (2026-08-15) puts it at **27.5×** — 268,066 tokens for a naive full-corpus read against ~9,763 per graph query.[5] The earlier ~36× figure that appeared here was an engineering estimate that had never been run against this repo, and is retired. The qualitative property holds regardless of the multiplier: answers arrive with `file:line` citations instead of synthesized guesses.

Plans that survive days, not turns (design property, with transcripts). The durable-plan spine is the difference between "the assistant forgot the plan after lunch" and a multi-day, dozens-of-turns execution that resumes itself on every Continue. This is MARVIN's most visible end-result in daily use, and the anonymized transcripts show it running on real work.[11]

Capabilities added since v0.1.55 (design properties). Four are worth naming because each extends a bet rather than decorating it. *Provider independence* — an OpenRouter BYOK path with provider-aware model resolution, so the assistant is not welded to one vendor's catalogue (ADR-0096). *A real terminal* — a persistent login shell on a pty, in-process, so `cd` persists, Ctrl-C interrupts, and a shell dies with the app rather than orphaning (ADR-0078). *Parallel implementation without shared state* — the single amendment to Bet 1: an implementer subagent bound to a git worktree MARVIN created may mutate *that* checkout, because a worktree removes the sharing that makes parallel agents dangerous (ADR-0081). *Blast radius before the edit* — directed call-graph queries answering "who calls this, at which line" and "what does this branch touch outside itself", which is Bet 2 applied to change rather than to search (ADR-0084, ADR-0085).

Guardrails that get measured, and corrected (worked example). The advisor originally persisted each caveat as its own backlog item. One session produced **12 items in about a minute; 10 were dismissed at the review and 2 kept** — the 10 were advice the executor had already acted on in the same turn, arriving pre-satisfied. The cost was never the writes; it was that two genuine blockers sat among ten dismissible ones. The design was amended the same day to one durable record plus one review item, with promotion deferred to the point where the user has the context to judge (ADR-0095). This is the intended failure mode of the whole approach: a deterministic contract produces a number, the number contradicts the design, and the design loses.

Rules that are mined, not written (measured, n=1 project). Every behavioural fix in MARVIN's history came from a human reading many transcripts by hand — the 2026-05-22 skill audit, the 2026-08-17 stall analysis, and a 2026-09-02 session audit that found `.gitlab-ci.yml`, a secrets-fetching script, a production sudoers grant and SMTP credentials shipped across **eight pushes with the review skills invoked 0×**, while every mechanical rule in the same session held. The immediate fix was a gate at `git commit`.[14] The general fix is the practice loop:[15] a nightly, read-only pass over a project's transcripts runs deterministic extractors for repeat failures and their paired successes (so a finding carries a *rate*, not a count), scores them with a small linear model whose weights are fit from the loop's own outcomes, proposes a rule at three distinct sessions, and — after the user approves — enforces it at the gate and measures whether it held: a recurrence is *regressed*, a quiet window is *confirmed*. No model reads a transcript; nothing changes MARVIN's behaviour without a click. The first backtest on a

real project read **397 sessions in 11 s** and produced six proposals; three were MARVIN's own defects and were fixed in the runtime that night — which is the intended shape. The loop is first a way of finding what to fix in MARVIN, and only second a way of constraining the model. The second report, three days later, was audited the same way: of four proposals, two were extractor defects fixed at the source, one was genuine, and one was a gate misfiring that is now instrumented rather than explained away. The same report carried the loop's newest kind, plan shape: **201 of 399 sessions** on that project had planned a change layer by layer, which is the failure the vertical-slice rule (§3) now refuses.[16]

Decisions that bind (design property). Architecture decision records written at decision time are re-read at the start of every future session and cross-checked during impact analysis. Month-eight work is confronted with month-two constraints mechanically, not by luck. A hundred and twenty ADRs govern MARVIN's own development — the tool is built under its own discipline, and several of its subsystems (the memory redesign, the context budget, the verify-then- remediate contract) exist because that discipline surfaced a real failure and forced a recorded fix.

A safety floor (design property, with one documented gap). The subagent read-only invariant, the hard-deny floor, checkpoint-based change review with revert, and the audit log together bound the blast radius of a single bad model decision — and every tool-channel mutation is attributable afterward. The gap is stated in §5: mutations made through shell commands are not checkpoint-snapshotted.

---

5. The trust model, concretely

For an engineer deciding whether to run an agentic tool on their machine, the security architecture is the evaluation. MARVIN's, in one place:

- **No backend.** All state local; inference direct to Anthropic; no MARVIN-operated service in the path. No telemetry by default.
- **Credentials.** The user's own API key — Anthropic Console, or OpenRouter — in a `0600` file on the user's disk. Never logged, never displayed beyond a last-four hint, never sent anywhere but the provider it belongs to. MARVIN does not use Claude.ai subscription logins: Anthropic's authentication policy reserves those for Claude Code and its own apps, and products built on the Agent SDK must use API keys.[9]
- **A structural gate, not a confirmation dialog.** Every tool call passes a classifier *before* execution: auto-allow (reads, whitelisted commands), confirm (edits, writes, unlisted shell), hard-deny (destructive patterns) — with the deny floor active even in full-bypass mode, and an audit trail either way.
- **Subagents cannot write.** Any tool call from a spawned agent that would mutate the workspace is hard-denied at the gate. This is the invariant that makes read-only research fan-outs safe.[4]
- **Only what MARVIN chose loads.** Turns load the user's skills and nothing else from Claude Code's settings: plugins the user enabled there are off, and another tool's memory is disabled. MARVIN's own in-process tool servers are trusted by the SDK's report of where a server came from, not by its name, so a configured server that borrowed a MARVIN name would confirm rather than run.[20]

- **Review is enforced at commit, not requested.** A `git commit` whose diff touches auth, credentials, CI, sudoers, `.env`, shell scripts or migrations is refused until both review skills have run this session; more than three files or fifty lines requires code review; docs and lockfiles pass. Two refusals per skill per turn, then allow and log — the gate cannot deadlock a session, and it fails open on git errors.[14]
- **Practice writes nothing on its own.** The loop that mines transcripts is plain code, runs read-only, and can only *propose*; a rule enforces only after the user approves it, a `deny` tier is offered only where a machine-checkable discharge exists, and every bypass is logged.[15]
- **Git is guarded at the argv level.** A single choke-point shells to git with `shell:false`, whitelists every ref and pathspec, rejects flag-injection vectors, and hard-denies force-pushes to main. Credential handling is inherited from the user's own helpers — MARVIN never prompts for, stores, or transforms git credentials.
- **Changes are reviewable and reversible — with one documented limit.** The gate snapshots each file's pre-image before the session first touches it; the review UI diffs against that snapshot, and reject reverse-applies — restoring the user's uncommitted state, which `git checkout` could not. The v1 limit, stated in the ADR and worth stating here: only tool-channel edits are pre-imaged. A mutation made *through a shell command* (`sed -i`, a codemod script) is not checkpoint-revertible — it is visible through git, but the snapshot mechanism does not cover it.
- **Signed releases.** Every release zip carries a minisign signature; the public key is pinned in the app repo, the tap repo, and the cask, so tampering with any one surface is visibly inconsistent with the other two.[10]

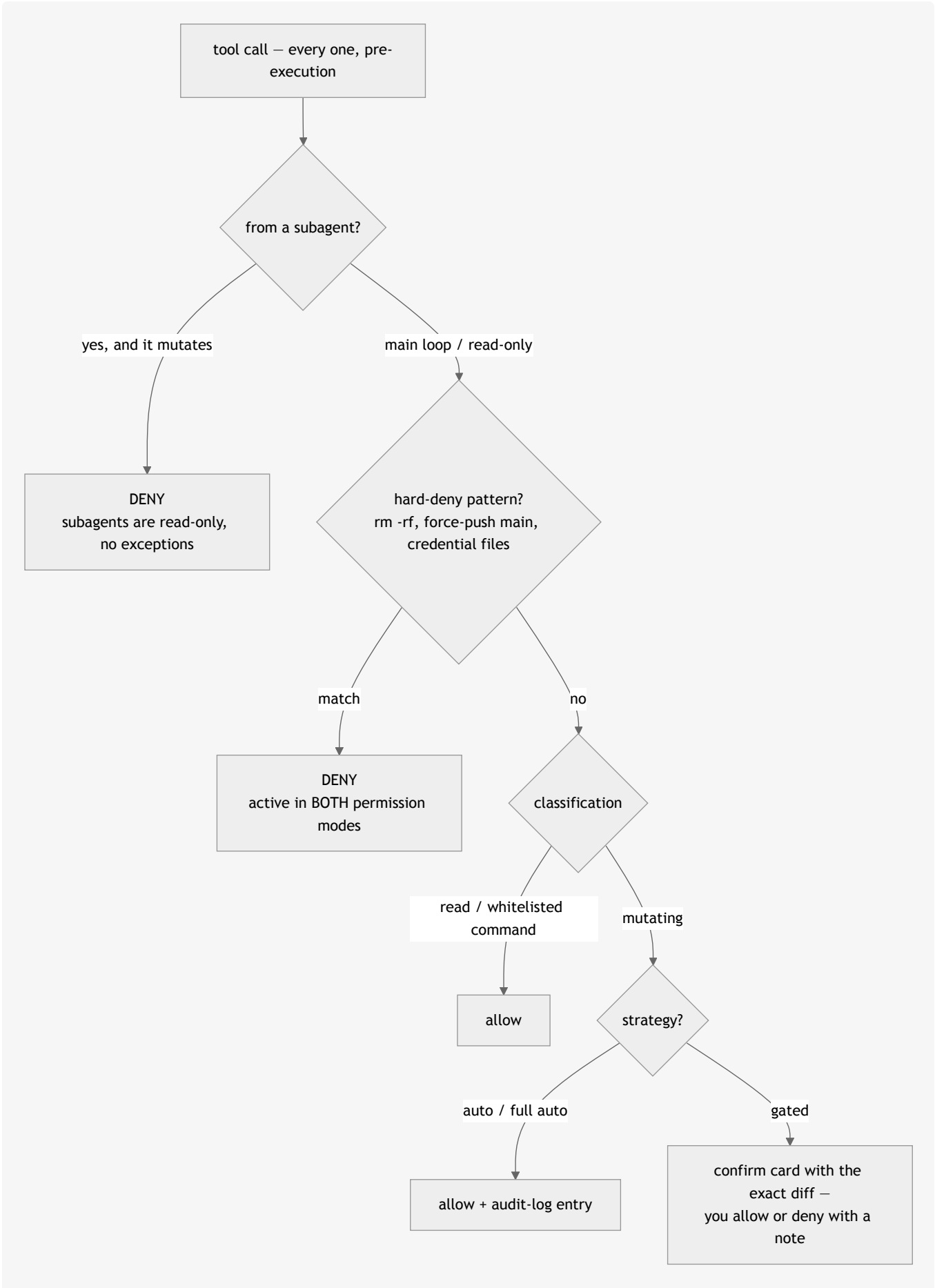

tool call — every one, pre-execution
from a subagent?
yes, and it mutates
main loop / read-only
DENY
subagents are read-only, no exceptions
hard-deny pattern?
rm -rf, force-push main, credential files
match
no
DENY
active in BOTH permission modes
classification
read / whitelisted command
mutating
allow
strategy?
auto / full auto
gated
allow + audit-log entry
confirm card with the exact diff — you allow or deny with a note

Figure 5 — the decision ladder every tool call descends. The deny floor and the subagent invariant sit above the auto/gated fork; no configuration weakens those two. The Bet-3 design hooks (graph-first, advisor-on-trigger, ship-review, and the practice rules the user accepted) run inside the classification step and can turn an otherwise-allowed call into a deny — they are the one layer with an off switch.

None of this makes an LLM infallible. It makes the *consequences* of fallibility bounded, visible, and — where the tool channel is used — reversible, and it documents the places it falls short instead of papering over them.

---

6. What MARVIN is not

Honest positioning, category by category:

- **Not an autocomplete tool.** MARVIN is a conversation-driven IDE, not an in-editor completion engine; it complements rather than replaces editor-native suggestion tools.
- **Not an agent swarm.** Tools built on parallel implementation agents promise throughput; the sequential-work evidence says they deliver error amplification. MARVIN's bet is the opposite: one accountable assistant, bounded read-only helpers, every write through one gate.
- **Not a cloud service.** There is no hosted MARVIN, no account, no server-side state, no usage dashboard watching the user. This costs MARVIN the features a backend enables (team sharing, cross-device sync) and buys the property that matters more to its audience: the code and the credentials stay home.
- **Not model-agnostic middleware.** MARVIN is built on the Claude Agent SDK and inherits its tool-use discipline; it routes different *roles* (planner vs. executor vs. advisor) to different Claude models, but it is not a thin wrapper over arbitrary LLM APIs.
- **Not free.** The discipline costs tokens: planning turns, advisor consults, and an 8-phase loop spend more per task than a bare CLI one-shot. The graph economics and the budgeted context are what claw the spend back; whether the trade nets out for you depends on how long your projects live. Short-lived scripts don't need MARVIN.
- **Not finished.** MARVIN is a young, opinionated, actively developed project (1.0 as of 2026-10-31 after the v0.1.x line; macOS/Apple Silicon only, ad-hoc signed, releases frequent). The 1.0 label marks the stability of the core loop and its gates, not a finished roadmap; it includes the supervisor conversation (ADR-0128), where no text is ever consent. The 128 ADRs are public; so are the audits that found real flaws — including the ones MARVIN's own tooling caught in its own repository.

The through-line: where the field bets on *more autonomy*, MARVIN bets on *more discipline* — encoded, wherever it has proven load-bearing, in mechanisms that don't depend on the model having a good day, and honestly labeled prompt-level where it hasn't yet.

---

7. Getting MARVIN

macOS (Apple Silicon), two commands, no toolchain:

```
brew tap RobertIlisei/marvin
brew install --cask marvin-ai
```

First launch requires macOS's one-time "Open Anyway" step (the app is ad-hoc signed — no paid developer program — and installs to `~/Applications`). MARVIN then needs an API key pasted in Settings: an Anthropic Console key, or an OpenRouter key.

- **Source & docs:** github.com/RobertIlisei/MARVIN
- **Worked examples of the modes and workflow:** `docs/guides/workflows.md`
- **Every functionality in depth:** the companion Technical Reference
- **Contributing:** the repo's ADRs (`docs/decisions/`) are the map of why things are the way they are; the roadmap (`docs/roadmap.md`) is the map of what's next. Both are the recommended first read.

Notes

1. The ≈70% / ≈17× figures come from a 2026 research pass across published multi-agent coding evaluations (work from Google, UIUC, Microsoft, and Anthropic Research), recorded in ADR-0001 as the project's founding design input — together with a failed multi-agent prototype of MARVIN itself. This paper reproduces them as the numbers the design bet on, not as results it independently verifies; the primary evidence offered here is the design and its source, not these figures.
2. Internal transcript audit, 2026-05-22: MARVIN session transcripts were scanned for turns matching each behavioral skill's own trigger conditions; five of six skills with "soft-nudge" prompt language had fired approximately zero times across thousands of qualifying turns. The finding drove the deterministic-trigger redesign.
3. The 8-phase workflow: `docs/concepts/eight-phase-workflow.md`.
4. Subagent read-only invariant: ADR-0030; advisor and scout: ADR-0007, ADR-0014.
5. Graph architecture and lifecycle: ADR-0028, ADR-0041. The 27.5× figure is `graphify benchmark` run on this repository (2026-08-15), replacing an earlier ~36× estimate that was never measured here. It is one repo's number, graph query vs. multi-file reading; treat it as an engineering heuristic, not a benchmark.
6. The firm-surfaces catalogue: `CLAUDE.md` ("The firm surfaces") and `sidecar/packages/runtime/src/personality.ts` in the repo.
7. Runtime design hooks: `sidecar/packages/runtime/src/design-hooks.ts` (`enforce` / `measure` / `off`).
8. Verify-then-remediate contract: v0.1.55 (2026-07-02), documented in the changelog and roadmap.
9. Auth resolution: `sidecar/packages/runtime/src/auth.ts`, `auth-config.ts`. Anthropic's authentication and credential-use policy (February 2026): Legal and compliance. The Keychain read of ADR-0029 predates that policy and is not a supported path.
10. Release signing: ADR-0026.
11. Both examples — the multi-day compliance plan and the graph-first root-cause — appear with full anonymized transcripts in `docs/guides/workflows.md`.

12. Memory as durable facts: ADR-0042; backlog: ADR-0044; backlog review, and why it reports rather than reconciles: ADR-0063; classification (`kind`, and `blocked` as its own axis): ADR-0064.
13. Context budget measurement: ADR-0041. Measured 2026-06-14 on one mature production project — not MARVIN's own repo — whose context comprised 139 ADRs plus a 419KB memory log (~566K tokens before the redesign; ~13.4K after). n=1; the mechanism, not the multiplier, is the claim.
14. Ship-review gate: ADR-0104. The 0× figure is from one audited session (25 turns, 2026-09-02); the hook is `checkShipReview` in `design-hooks.ts`.
15. Practice loop: ADR-0105; user guide: `docs/guides/practice.md`. The design follows Anthropic's own guidance that memory files are "context, not enforced configuration" and that blocking an action takes a hook (Claude Code docs: memory), and two 2026 papers: *Learning What to Remember* (arXiv:2606.12945) — a linear value model whose weights are learned from downstream outcomes retained 77 % of the evidence that later mattered, against 66 % for uniform weights — and *MemGuard* (arXiv:2608.21867), which keeps a verifier's signals attached to a memory for its lifetime. The 397 sessions / 11 s backtest is one project on one machine.
16. Vertical slices: the Phase 5 rule in `personality.ts`, the `relations` filter on `graph_path`, and the `plan.horizontal` / `plan.vertical` kinds (extractor v6). Layers are discovered by `discoverAreas` in `graphify-bridge/src/plan-areas.ts`: code files grouped by directory, split where a directory holds two or more substantial groups, each area's vocabulary its own directory names and file stems kept where distinctive; a milestone crosses layers only on directory-level evidence from two areas. The tracer-bullet framing follows Matt Pocock's AI Engineer Europe talk on why agents fail on horizontal plans. The 931 / 318 / 201 figures are a dry run over one project's 399 transcripts before the rule shipped.
17. Context budget, second pass: ADR-0118 (one prompt per session, settings isolation, indexes as a recent slice, deferred servers, the ~4.4K base prompt and `core-skills.ts`); measured with the SDK's `getContextUsage` via `sidecar/scripts/context-baseline.ts` on one production project, 2026-09-21. Instruction files: ADR-0119 (a near-copy is ≥ 50 % of its lines in the other file; real copies measured 85–88 %, a distinct file 9 %).
18. Own-change consequences: ADR-0121 (`cross-layer-refs.ts`, `checkShipImpactRequired`, `ownChangeConsequences`). The textual pass is labelled a heuristic: it matches token shapes, never a stack, and was tuned on three live branches.
19. Full auto: ADR-0115. Stalls: ADR-0120 (`keep-awake.ts`, `checkSearchWithoutPath`, `MAX_JOB_DONE_DEFERRALS`); the 51-of-51 figure is `pmset -g log` against one 20-hour transcript.
20. MCP trust by provenance: ADR-0117. Defence in depth — no reachable spoof existed; missing provenance keeps the name-based allow and is logged.

© 2026 Robert Ilisei. MARVIN is open source (MIT). This paper describes v1.0.0; the repository is the authoritative, current reference.